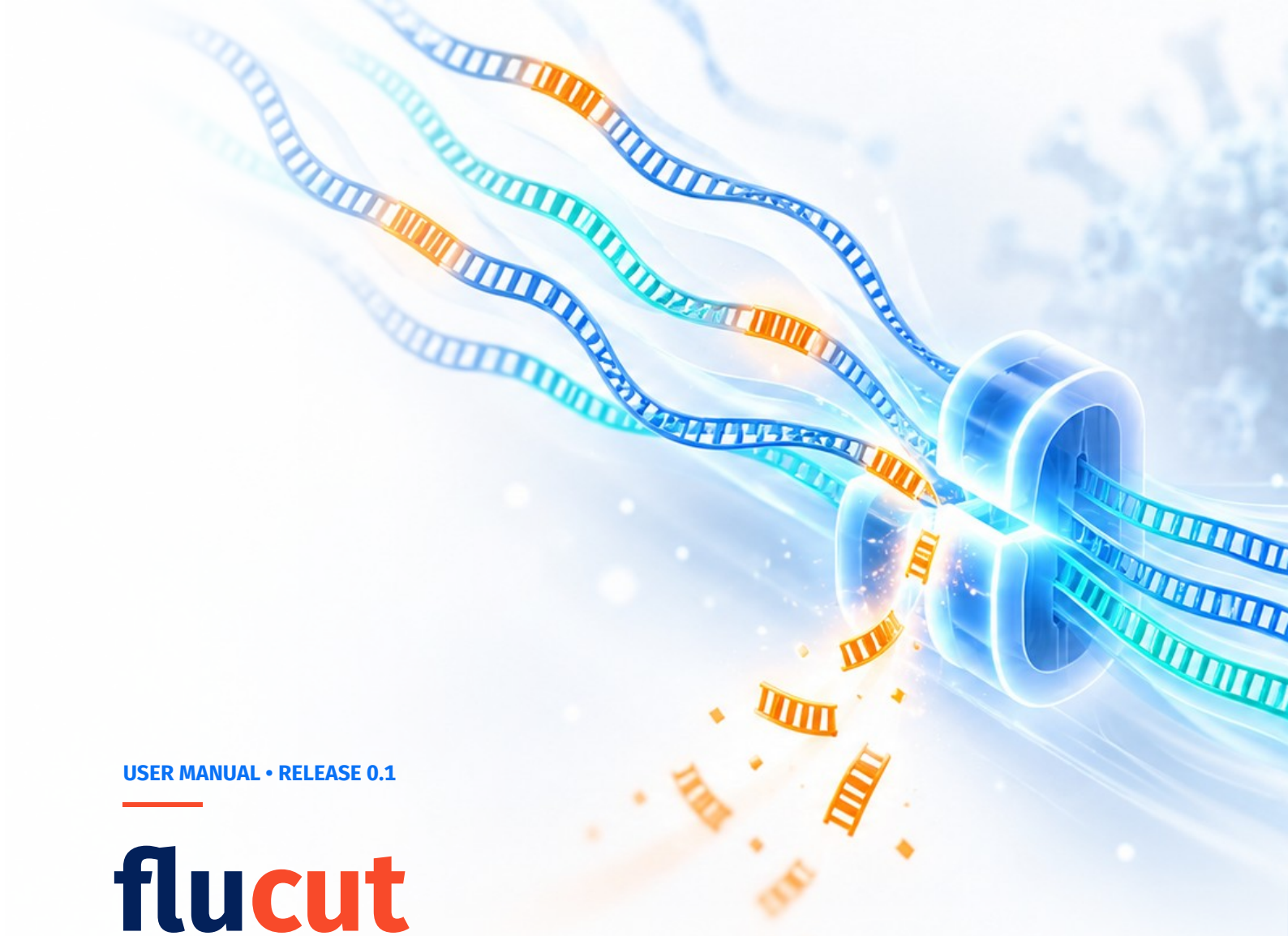


USER MANUAL • RELEASE 0.1

flucut

Primer trimming for influenza A before the assembly with IRMA

Uni12/Uni13 amplicons, internal priming

Giorgio Gonnella
BAIA, Institut Pasteur du Cambodge

October 30, 2026

Contents

1 Introduction **3**
- 1.1 What flucut does . . . 3
- 1.2 Why it is needed . . . 3
- 1.3 What it does not do . . . 4
- 1.4 Platforms . . . 4

2 Installation **5**
- 2.1 From the source . . . 5
- 2.2 The tests . . . 5
- 2.3 Packages . . . 5

3 Quick start **7**

4 The primers **9**
- 4.1 The conserved termini . . . 9
- 4.2 The built-in primers . . . 9
- 4.3 The schemes . . . 10

5 How it works **11**
- 5.1 The layout of a read . . . 11
- 5.2 The search of a primer . . . 11
- 5.3 The window . . . 12
- 5.4 The trimming . . . 12
- 5.5 Threads . . . 12

6 Internal priming **13**
- 6.1 The artefact . . . 13
- 6.2 What flucut does about it . . . 13
- 6.3 Classification against references . . . 14
- 6.4 The length check . . . 14

7 Outputs **15**
- 7.1 The trimmed reads . . . 15
- 7.2 summary.tsv . . . 15
- 7.3 lengths.tsv . . . 16
- 7.4 Messages . . . 16

8 Using the reads with IRMA **17**
8.1 What was checked . . . 17

9 Performance and accuracy **19**
9.1 The consensus . . . 19
9.2 The reads . . . 19
9.3 Speed . . . 20

10 Troubleshooting **21**
10.1 Errors in the command line . . . 21
10.2 Errors in the inputs . . . 21
10.3 Messages about the reads . . . 22

11 Command line reference **23**
11.1 Arguments . . . 23
11.2 Options . . . 23
11.3 Classification against the references . . . 24
11.4 Other options . . . 24

12 Glossary **25**

13 About **27**
13.1 Use of AI tools . . . 27

flucut removes the primers of the influenza A multisegment RT-PCR (the universal Uni12 and Uni13 primers) from sequencing reads, before the assembly of the segments, e.g. with IRMA. It finds the primers at both ends of each read, in both orientations, also when a read begins inside a primer, and removes them by position. It recognises the reads started by a primer at an internal site of a segment (internal priming), whose primer bases otherwise reach the consensus as false variants, such as the I348K change of the H5 hemagglutinin.

This manual explains how to install and use flucut, from a first run on the test data of the repository to the classification of the reads against reference sequences. The chapters can be read in order, as a tutorial, or used as a reference: the last chapters list every message and every option.

1 Introduction

1.1 What flucut does

The whole genome of influenza A is often sequenced from a multisegment RT-PCR: two universal primers, Uni12 and Uni13, bind the conserved termini of the eight segments and amplify all of them in one reaction. The bases of a read which lie under a primer come from the primer, not from the virus: a difference between the virus and the primer at such a position is overwritten. flucut removes these bases from the reads, before the assembly.

For each read, flucut:

- finds the primers at both ends of the read, in either orientation, with sequencing errors, also when the read begins or ends inside a primer;
- removes every primer-derived base and everything outside the primers (adapter, barcode and other flanking bases), by position;
- writes the rest of the read, in its original orientation, to a gzipped FASTQ file for each sample, with tags in the title telling what was found and removed;
- with reference sequences of the segments, tells whether each end of the read is at a terminus of a segment or at an interior position (internal priming), and reports it.

1.2 Why it is needed

The untailed Uni12 primer is only twelve bases long, with a degenerate base. Sequences similar enough to anneal to it occur inside the segments: the primer can start an amplicon in the middle of a segment, and the reads of such an amplicon carry the primer's bases where the template has others. In the H5 hemagglutinin (HA) of Cambodian H5N1 viruses this changes the isoleucine at position 348 to a lysine (I348K): a false variant, reproducible from run to run (see *Internal priming*). Removing the primers from the reads removes it.

IRMA, the assembler used for influenza by many laboratories, does not remove primers: it matches a single adapter sequence, and its pipeline MIRA does no primer trimming for influenza Nanopore reads.

1.3 What it does not do

- It does not assign the reads to samples (demultiplexing): give it the reads of one sample (a file, or a directory of files, e.g. a barcode directory written by the basecaller).
- It does not assemble the segments or call variants: that is the work of IRMA or another assembler.
- It knows the primers of the influenza A multisegment RT-PCR (*The primers*); other amplicon schemes are not supported.

1.4 Platforms

flucut is not specific to a sequencing platform: the errors allowed in a primer are an option. The defaults are those of Oxford Nanopore reads.

2 Installation

2.1 From the source

Only a C compiler (C11 with GNU extensions) and `make` are needed: the libraries used (libcbaia, edlib, libyaml, zlib) are included in the repository, in `ext`.

```
git clone https://github.com/baia-ipc/flucut
cd flucut
make
make test
```

The program is `bin/flucut`. `make install` copies it to `/usr/local/bin` (`make install PREFIX=<dir>` to another prefix).

`make SYSTEM_LIBS=1` uses the zlib and libyaml of the system instead of the included copies.

2.2 The tests

`make test` runs the unit tests and the program tests under valgrind (if it is installed), and the check of the coding style. The program tests run flucut on the files of `testdata` and compare the results with `testdata/expected`.

2.3 Packages

There is no Bioconda package yet.

3 Quick start

The repository contains a small test set in `testdata`: twelve reads simulated from a real HA record (NCBI `NC_004908.1`), each one a typical situation (see `testdata/README.md`). From the root of the repository, after `make`:

```
bin/flucut -o out testdata/reads.fastq
```

flucut writes:

```
SUCCESS | out/reads.fastq.gz: 11 of 12 reads written (10 with both
  primers, 1 with one, 1 with none; 1 too short, of which 1 dimers)
SUCCESS | out/reads.fastq.gz: complete amplicons, lengths between the
  primers (peaks): 1700-1709 (5) 1000-1009 (2) 20-29 (1) 900-909 (1)
```

(the lines are wrapped here, without the time stamps). The output directory contains:

`reads.fastq.gz`
: the trimmed reads, ready for IRMA;

`summary.tsv`
: the counts of the reads;

`lengths.tsv`
: the lengths of the complete amplicons.

The first read, an amplicon with both primers, became:

```
@full_plus simulated  fs:Z:+  fl:Z:uni12r  fr:Z:mbtuni-13  tl:i:12  tr:i:23
GGAATTACTTAACTAGCAAAATGGAAACAATATCACTAATAACTATACTACTAG...
```

The tags tell that the read is of the plus strand of the amplicon (`fs`), that the primer `uni12r` was found at its left end and `mbtuni-13` at its right end, and that 12 and 23 bases were removed at its start and end (*Outputs*). The primer dimer (two primers and nothing between them) was not written.

With the reference of the segment, flucut also tells where each read started:

```
bin/flucut -o out2 -r testdata/ref-ha.fasta testdata/reads.fastq
```

It reports the ends started by internal priming and the complete amplicons much shorter than the reference (the figure shows this run in a terminal, where flucut shows its banner first), and the read simulated as started inside the segment, at position 712, is tagged as such:

```
@internal12_plus simulated  fs:Z:+  fl:Z:uni12r  ol:Z:internal
  ml:Z:NC_004908.1:712:+  fr:Z:mbtuni-13  or:Z:terminal
  mr:Z:NC_004908.1:1714:-  tl:i:12  tr:i:23
```

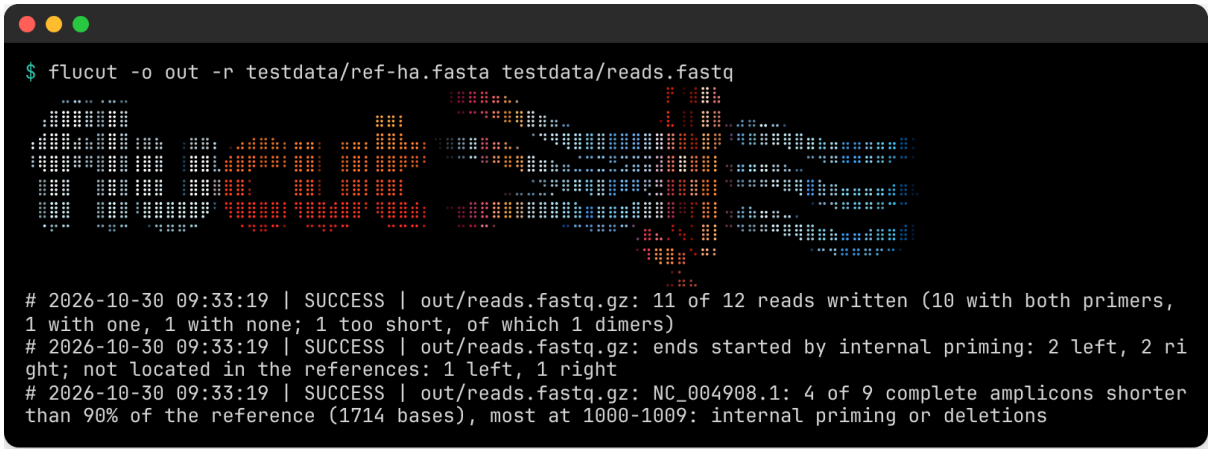


Fig. 1: `flucut -o out -r testdata/ref-ha.fasta testdata/reads.fastq` in a terminal of 100 columns (`terminal.js`).

To trim a real run, give the reads of each sample, as files or as directories (one output per input):

```
flucut -o trimmed barcode01/ barcode02/ barcode03/
```

and give the outputs to IRMA (*Using the reads with IRMA*).

4 The primers

4.1 The conserved termini

All influenza A segments end in the same conserved sequences: the 12 nucleotides of one end and the 13 of the other are (almost) identical in all eight segments and in all subtypes. On the plus strand of an amplicon (the orientation of the mRNA) they are:

```
AGCAAAAGCAGG ... segment ... CCTTGTTTCTACT
```

The universal primers bind them: Uni12 (`AGCAAAAGCAGG`) at the left end, Uni13 (`AGTAGAAACAAGG`, the reverse complement of the right end) at the right end. The fourth base of the 12-nucleotide terminus is A in some segments and G in others; the degenerate form of Uni12, `AGCRAAAGCAGG` (R = A or G), matches both.

4.2 The built-in primers

flucut knows the primers by name; their sequences are quoted from the publications, each one documented in `src/primers.h`.

name	sequence	source
uni12	`AGCAAAAGCAGG`	the conserved terminus (Uni12 of Hoffmann et al. 2001, as quoted by Zhou et al. 2009)
uni12r	`AGCRAAAGCAGG`	the same, degenerate fourth base
uni13	`AGTAGAAACAAGG`	the conserved terminus (Uni13)
mbtuni-12	`ACGCGTGATCAGCAAAAGCAGG`	Zhou et al. 2009, J Virol 83:10309-10313
mbtuni-12.4r	`ACGCGTGATCAGCRAAAGCAGG`	Licheri et al. 2025, Genome Med 17:103
mbtuni-13	`ACGCGTGATCAGTAGAAACAAGG`	Zhou et al. 2009

The tailed primers carry the 5' tail `ACGCGTGATCAG`, which overlaps the terminus by two bases: flucut stores each primer whole, with the position at which its conserved terminus begins, and knows which bases of a read come from the tail and which from the terminus.

4.3 The schemes

A scheme names the primers expected at each end of an amplicon (option `-s`):

scheme	left end (Uni12 side)	right end (Uni13)
uni12-mbt13	uni12r	mbtuni-13
mbt	mbtuni-12, mbtuni-12.4r	mbtuni-13
uni	uni12, uni12r	uni13

`uni12-mbt13` is the default: the untailed degenerate Uni12 primer and the tailed Uni13 primer, the combination used at the Institut Pasteur du Cambodge. When a scheme has several primers at an end, the best match is taken.

5 How it works

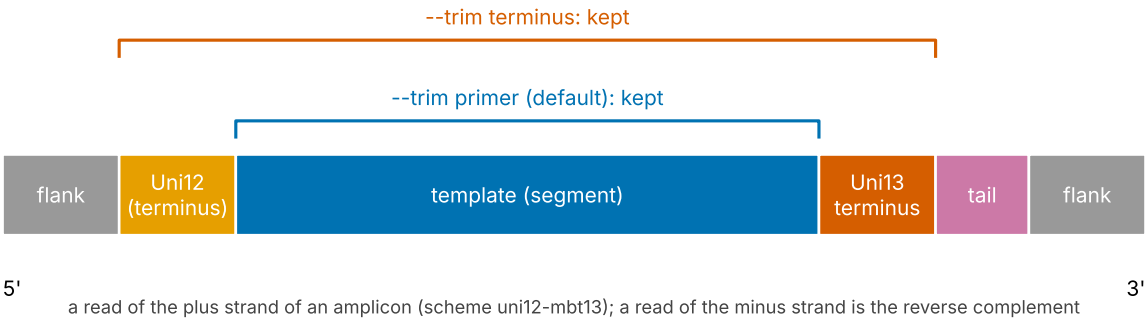


Fig. 1: The parts of a read of an amplicon, and what each trimming policy keeps (`make_diagrams.py`).

5.1 The layout of a read

A read of an amplicon contains, from one end to the other: flanking bases (adapter, barcode, or bases of the library preparation), the primer of one end, the template (the segment), the primer of the other end, and flanking bases again. It can be sequenced in either orientation, and it can begin or end inside a primer.

flucut searches the primers of the left end of the scheme at the start of the read and those of the right end at its end. If the read was sequenced in the other orientation, the primers are found in its reverse complement; the positions are always reported in the original read, which is not reverse-complemented.

5.2 The search of a primer

A primer is aligned to the read with edit distance (substitutions, insertions and deletions), allowing `-e` errors (default 0.2 of the length of the primer: 2 errors in Uni12). Degenerate bases (e.g. the R of Uni12) match any of the bases they stand for, in the primer and in the read (e.g. N).

At an end of the read, the primer can be partial:

- it can overhang the read, when the read begins inside the primer: the missing part is not counted as errors, and up to half of the primer can be missing;

- it can lack up to half of its outer bases after some flanking bases, when the amplicon was shortened before the adapter was ligated; this shorter form must match with one error less than its length allows, as a shorter sequence matches by chance more easily.

5.3 The window

A match of a primer deep inside a read is not the primer which started the amplicon: trimming there would remove template. The primer of an end is therefore accepted only if it begins (left end) or ends (right end) within `-w` bases of that end of the read (default 100: enough for the barcoding adapters of Oxford Nanopore, 46 bases, and further flanking bases).

Within the window, the outermost match is taken: the primer which started the amplicon is the one nearest to the end of the read, and a primer-like sequence further inside is template. Matches of a primer between the two end primers are counted (tag `ni`), not trimmed: they are the sites at which a primer can anneal, and the junctions of chimeras.

5.4 The trimming

The removal is by position: the whole span of the primer alignment is removed, whether or not the bases under it differ from the primer, and everything outside it.

`--trim primer` (the default)
: removes every primer-derived base, tail and conserved terminus. No base of the output comes from a primer. The consensus is shorter than the segment by the termini (12 and 13 bases): those bases would have been the primer's, not measured sequence.

`--trim terminus`
: removes the tail and everything outside the primer, and keeps the bases of the conserved terminus, so that the consensus has complete ends. The kept bases are those of the primer: with the degenerate Uni12, the fourth base is a mix of A and G; where the segment differs from the primer (e.g. the 3' end of segment 2 of A/Puerto Rico/8/1934, `CCTTGTTCCTACT`), the primer's base is kept. At an internal priming site, the primer bases are not the template at all: this policy therefore requires the references (`-r`), against which such ends are recognised and trimmed whole (*Internal priming*).

Reads with nothing left (primer dimers) and reads shorter than `-m` bases after the trimming are not written.

5.5 Threads

The reads are read in batches; each batch is split between the threads (`-j`), each of which trims and compresses its part, and the parts are written in the order of the input: the output is the same for any number of threads.

6 Internal priming

6.1 The artefact

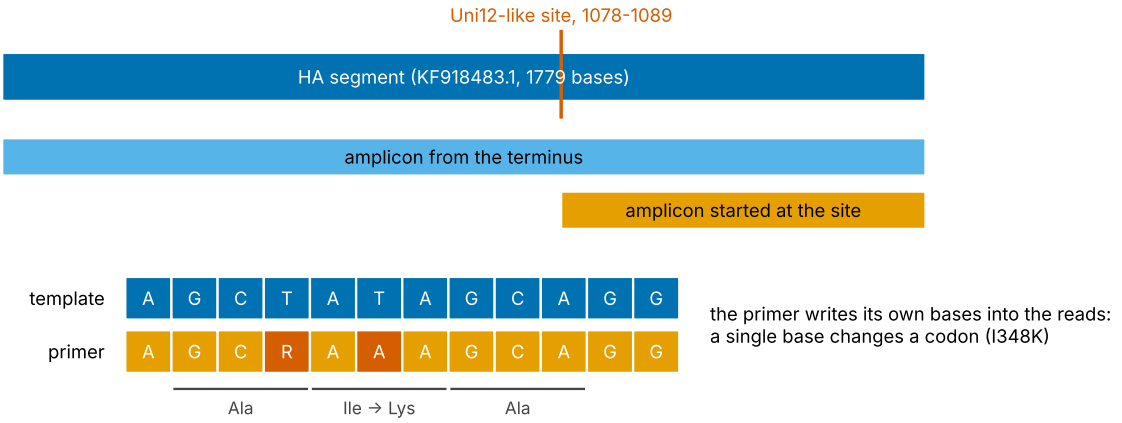


Fig. 1: The Uni12-like site of the HA of A/chicken/Cambodia/X0815302/2013 (H5N1), NCBI KF918483.1, and the amplicon the primer starts there (`make_diagrams.py`).

The untailed Uni12 primer is twelve bases long, with a degenerate base. A sequence similar enough to it occurs inside the HA segment of the H5N1 viruses of Cambodia: after the codons of the motif GLFGAIAG, the bases `AGCTATAGCAGG` differ from the primer `AGCRAAAGCAGG` at two positions. The primer can anneal there and start an amplicon in the middle of the segment. Its reads begin with the bases of the primer: the R falls on the third base of an alanine codon (silent), but the A replaces the T of the isoleucine codon ATA, which becomes the lysine codon AAA. If enough reads carry it, the consensus shows I348K, a variant which looks real and is not. All the 30 Cambodian H5N1 HA sequences of GenBank checked for the benchmark have this site.

6.2 What flucut does about it

The primer bases of an internally primed read are at its end, like those of any other read: removing every primer (`--trim primer`, the default) removes the artefact, without any reference. The site is still covered by the reads of the complete amplicons, which carry the template.

In the benchmark (*Performance and accuracy*), the untrimmed simulated reads give AAA at the site in

the IRMA consensus, and the reads trimmed by flucut give ATA, the truth.

6.3 Classification against references

With reference sequences of the segments (`-r`, a FASTA file), flucut also tells where each read started. The bases which follow the primer of an end (the probe: `--probe-len`, default 32 bases; for the right end, the reverse complement of the bases before its primer) are located in the references:

- at a terminus of a segment (within `--terminal-window` bases of it, default 60, as references often lack some bases of the termini): a normal amplicon (tags `ol`/`or`: `terminal`);
- at an interior position: internal priming (`internal`); the tags `ml`/`mr` give the reference and the position;
- not located (no reference of the subtype, a probe too short): `unknown`.

Among equally good locations, the one at a terminus is taken: an ambiguous read is never classified as internally primed.

The references are indexed by their k-mers (10-mers), so that the time hardly depends on their number; a probe without a candidate is aligned to all of them. `--exhaustive-search` aligns every probe to all of them, to check the index (the outputs were identical in the benchmark).

With `--trim primer` the classification is reported only. With `--trim terminus`, an end classified as internal loses its whole primer, and the others keep their terminus.

`scripts/fetch_references.py` downloads segment references from NCBI, with their provenance.

6.4 The length check

The complete amplicons of a segment (both primers found) have the length of the segment. Internal priming, or a deletion (e.g. in the defective interfering RNAs of the polymerase segments), shows as a group of shorter amplicons. flucut writes their lengths to `lengths.tsv` (*Outputs*):

- with references, for each reference the amplicons shorter than 90% of its length are counted (`short_complete` in `summary.tsv`) and reported, with the length where most of them are. The reference length is the measure, not the most frequent length, as the internally primed amplicons can be the majority;
- without references, the peaks of the lengths are listed.

In the benchmark, 209 of the 340 complete HA amplicons were reported as short, most at 670-679 bases: the length from the internal site to the end of the segment.

7 Outputs

For each input (a file of reads, or a directory of files of reads), flucut writes `<outdir>/<name>.fastq.gz`; `<name>` is the name of the file without its suffix (`.fastq`, `.fq`, optionally `.gz`, or `.bam`), or that of the directory. Two inputs with the same name are an error, as one would overwrite the other. `summary.tsv` and `lengths.tsv` have one part for each input.

7.1 The trimmed reads

`<name>.fastq.gz` (gzip, FASTQ) contains the reads which are not empty and not shorter than `-m` bases after the trimming, in the order of the input, in their original orientation, with their quality values cut as the sequence. Their title is the original title followed by tags in the SAM text format, separated by tabs (as in the FASTQ written by Dorado), unless `--no-tags` is given:

tag	meaning
`fs`	strand of the amplicon: +, -, or `.` (no primer)
`fl`	the primer found at the left end (Uni12 side)
`fr`	the primer found at the right end (Uni13 side)
`tl`	bases removed at the start of the read
`tr`	bases removed at the end of the read
`ni`	primer matches inside the read (if any)
`ol`	with `-r`: origin of the left end (`terminal`, `internal` or `unknown`)
`or`	the same for the right end
`ml`	with `-r`: where the probe of the left end mapped, `<reference>:<position>:<strand>` (1-based position on the plus strand)
`mr`	the same for the right end

The left and right ends are those of the amplicon: for a read of the minus strand, the left end is at the end of the read. IRMA accepts the tags (*Using the reads with IRMA*).

7.2 summary.tsv

One line for each input, tab-separated, with a header:

column	meaning
`output`	the name of the input
`reads_in`	reads read
`bases_in`	their bases
`reads_out`	reads written
`bases_out`	their bases
`both_ends`	reads with the primers of both ends
`one_end`	reads with the primer of one end
`no_end`	reads without primers
`minus_strand`	reads found on the minus strand of the amplicon
`dimers`	primer dimers (both primers, nothing between)
`too_short`	reads not written: empty or shorter than `-m` (dimers included)
`with_internal_match`	reads with primer matches inside them
`left_terminal`, `left_internal`, `left_unknown`, `right_terminal`, `right_internal`, `right_unknown`	with `-r`: the left and right ends found, by origin; `NA` without references
`short_complete`	with `-r`: complete amplicons shorter than 90% of their reference; `NA` without references

7.3 lengths.tsv

The lengths of the complete amplicons (both primers found; the template between the two primers), in bins of 10 bases, tab-separated, with a header: `output`, `group` (the reference to which the probe of the left end, or else of the right end, mapped; `not_located`; or `all` without references), `length` (the start of the bin) and `reads`. Only the bins with reads are written; the last bin collects the lengths from 4000 bases on.

7.4 Messages

The messages go to the standard error (or to the file of `-L`): the counts of each input, the internal priming found, and the length check (*Internal priming*). In a terminal, the banner of flucut is shown first (not with `-q` or `-L`).

8 Using the reads with IRMA

The trimmed reads of each sample are the input of IRMA, e.g. with the configuration for Nanopore reads:

```
IRMA FLU-minion trimmed/barcode01.fastq.gz sample01
```

8.1 What was checked

These points were measured with IRMA 1.4.0 (FLU-minion) on simulated reads (`benchmark/RESULTS.md`):

- **The tags.** IRMA accepts the tags in the titles: the consensus was the same with and without them. `--no-tags` writes the titles unchanged, if another program needs it.
- **The ends of the consensus.** With FLU-minion as it is, IRMA does not extend the consensus beyond its own references: 20 to 58 bases are missing at each end of the segments, with or without trimming. With the reference elongation (an external configuration file with `SKIP_E=0`, given with `-c`), the consensus of the reads trimmed by flucut ends exactly where the primers began: 12 bases short at one end and 13 at the other, without errors. With untrimmed reads, the consensus extends into the flanks, with insertions.
- **Keeping the termini.** With `--trim terminus` and the elongation, the consensus has complete ends, but the bases there are the primer's: the fourth base is an ambiguity (R) in all segments, and where a segment differs from the primer, the primer's base is kept.
- **Low coverage.** FLU-minion replaces with N the bases supported by fewer than 50 reads (`MIN_CONS_SUPPORT`) in the amended consensus: with fewer reads, the amended consensus is all N, whatever the trimming; the majority consensus (`A_<gene>.fasta`) is still written.

The configuration file for the elongation:

```
echo 'SKIP_E=0' > elong.sh
IRMA FLU-minion trimmed/barcode01.fastq.gz sample01 -c elong.sh
```

9 Performance and accuracy

The numbers of this chapter come from the benchmark of the repository (`benchmark/`): what was run, on which data, with which versions and commands, is in `benchmark/RESULTS.md`; the scripts are in `benchmark/`. The reads are simulated, so that the origin of every base is known: amplicons of a genome made of the segments of A/Puerto Rico/8/1934 (H1N1) and the HA of A/chicken/Cambodia/X0815302/2013 (H5N1), which carries the internal priming site of the I348K artefact, with Nanopore-like errors (`benchmark/README.md`). No real run has been measured yet.

9.1 The consensus

On 3200 simulated reads (400 per segment, 60% of the HA reads started at the internal site), assembled by IRMA 1.4.0 (FLU-minion):

- the untrimmed reads give the artefact: AAA (lysine) at the I348K codon of the majority consensus; the reads trimmed by flucut give ATA, the truth;
- with the reference elongation of IRMA, the consensus of the reads trimmed by flucut has no error, and ends 12 and 13 bases short of the segments (the primers); with `--trim terminus` it has complete ends, with the primer's bases there (*Using the reads with IRMA*);
- at 10, 20 and 40 reads per segment (three simulations each), the untrimmed reads give the artefact in 5 of 9 runs, the reads trimmed by flucut in none.

9.2 The reads

Bases of the 3200 reads by their true origin, after the trimming:

tool	template removed	primer kept	flank kept	internal site kept
untrimmed	0	102554	127032	2698
flucut	882	1852	7557	23
cutadapt	102408	3037	8784	25

cutadapt (`-g` and `-a`, not anchored, as the reads have flanking bases) removes everything before a primer-like sequence found anywhere in a read: with 2 errors, Uni12 also matches inside the segments, e.g. at the I348K site, and complete HA reads are cut there. Its consensus is nonetheless the same as that of flucut in these simulations, also at low coverage. The comparison with the other tools (IRMA-core trimmer, BBDuk, Porechop_ABI, dorado, Barbell, samtools ampliconclip) is in `benchmark/RESULTS.md`.

9.3 Speed

flucut trimmed the 3200 reads (5.3 million bases) in 1.3 seconds with one thread. The classification against references, through the k-mer index, adds little: 0.8 s instead of 0.7 s with 8 references, 1.1 s with 80 (measured before the search of the truncated primers; with the exhaustive search, 2.0 and 14-16 s).

10 Troubleshooting

Each message of flucut, its cause and what to do. Errors end the program with exit status 1.

10.1 Errors in the command line

`--trim terminus requires --references`
: Keeping the termini without references would keep the primer bases of the internally primed reads (the artefact). Give the references (`-r`), or use the default `--trim primer`.

`unknown scheme '<name>'`
: The name given to `-s` is not a built-in scheme; the help lists them.

`'primer' or 'terminus' expected`
: The value of `-T` is neither.

`a fraction below 1 or a whole number of bases expected`
: The value of `-e` is negative, not a number, or a fraction above 1 (e.g. 1.5).

`argument must be an integer ...`
: A value of `-w`, `-m` or `-j` out of its range.

`Option '--probe-len' / '--probe-errors' / '--terminal-window' ...`
: A value out of its range (see *Command line reference*).

`the output directory is empty`
: `-o ""` was given.

10.2 Errors in the inputs

`<file>:  not a file of reads`
: An input which is not a directory must end in `.fastq`, `.fq` (optionally `.gz`) or `.bam`.

`<dir>:  no file of reads in the directory`
: A directory given as input contains no file with these suffixes.

`<a> and <b>:  both would be written to <name>.fastq.gz`
: Two inputs have the same name (e.g. two directories `barcode01` in different runs): trim them in separate runs, with different output directories.

<file>: the input would be overwritten by the output
The output directory is the directory of an input with the same name: choose another output directory.

Errors in the references file
`the sequence of "<name>" is empty`, `no sequence found`, `invalid base`, `sequence before the first header line`: the file given to `-r` is not a valid FASTA file.

10.3 Messages about the reads

not located in the references: <n> left, <n> right
Read ends whose probe was not found in the references. A few are expected (reads with many errors, dimers); many mean that the references do not fit the run (e.g. a missing subtype): download references of the subtypes of the run.

<reference>: <n> of <m> complete amplicons shorter than 90% of the reference
Complete amplicons much shorter than their segment: internal priming or deletions (*Internal priming*). Look at the reads of `lengths.tsv` and at the `ml`/`mr` tags.

Many reads without primers (no_end in summary.tsv)
The scheme does not match the primers of the run (`-s`), the reads were already trimmed (e.g. by the basecaller), or the error rate of the reads is higher than `-e` allows.

11 Command line reference

```
flucut [options] <input>...
```

11.1 Arguments

<input>
: A FASTQ file (optionally gzipped), an unaligned BAM file, or a directory of such files (recognised by the suffixes `.fastq`, `.fq`, optionally followed by `.gz`, and `.bam`; e.g. a barcode directory of a MinKNOW run, whose files are trimmed into one output). Each input gives one output (*Outputs*).

11.2 Options

-o, --outdir <dir>
: The output directory, created if needed. Default: `flucut-output`.

-s, --scheme <name>
: The primer scheme (*The primers*): `uni12-mbt13` (default), `mbt` or `uni`.

-T, --trim <policy>
: `primer` (default): remove every primer-derived base. `terminus`: keep the conserved termini; requires `--references` (*How it works*).

-r, --references <file>
: A FASTA file of segment references: the read ends are classified as terminal or internally primed (*Internal priming*). With `--trim primer` the classification is only reported; with `--trim terminus` it decides the trimming of the internal ends. Default: none.

-e, --max-errors <x>
: The errors allowed in a primer: a whole number of bases, or, below 1, a fraction of the length of the primer. Default: 0.2.

-w, --window <n>
: A primer is accepted at an end only if it begins (left end) or ends (right end) within <n> bases of that end of the read. Default: 100.

-m, --min-length <n>
: Reads shorter than <n> bases after the trimming are not written. Default: 1 (empty reads, e.g. primer dimers, are never written).

-j, --threads <n>
The number of threads (1 to 256). The output does not depend on it. Default: 4.

--no-tags
Do not append the tags to the read titles.

11.3 Classification against the references

These options are used with `--references`.

--probe-len <n>
The number of bases after the primer of an end located in the references. At least 16. Default: 32.

--probe-errors <x>
The fraction of the probe allowed to be errors (0 to below 1). Default: 0.2.

--terminal-window <n>
A probe mapped within <n> bases of an end of a segment is terminal. Default: 60.

--exhaustive-search
Align each probe to the whole of every reference, without the k-mer index (slower; for checking the index).

11.4 Other options

-q, --quiet
Only error messages.

-v, --verbose
Additional information (e.g. the values of the options).

-D, --debug
Debugging information.

-L, --logfile <file>
The messages to a file instead of the standard error.

-h, --help
The help, with the list of the schemes.

-V, --version
The version.

Only one of `-q`, `-v` and `-D` can be given. Invalid values of the options are errors (exit status 1), reported with the usage.

12 Glossary

amplicon
: A molecule made by the PCR: here a segment, or part of it, between the two primers.

complete amplicon
: A read in which the primers of both ends were found.

conserved terminus
: The 12 or 13 bases at the ends of the influenza A segments, the same in all segments and subtypes, to which the universal primers bind.

flank
: The bases of a read outside the primers: adapter, barcode, and other bases of the library preparation.

internal priming
: The start of an amplicon by a primer annealed at an interior position of a segment, whose bases then replace those of the template in the reads.

probe
: The bases of a read after the primer of an end, located in the references to classify the end.

scheme
: The primers expected at the two ends of an amplicon.

tail
: The bases of a primer before its conserved terminus, added by the protocol (e.g. `ACGCGTGATCAG` of the mbtuni primers).

window
: The part of a read, at each end, in which the primer of that end is accepted (`-w`).

13 About

flucut is developed by Giorgio Gonnella at BAIA, Institut Pasteur du Cambodge. It is free software under the ISC license (`LICENSE`).

The source code: https://github.com/baia-ipc/flucut

flucut includes libcbaia (ISC), edlib (MIT), libyaml (MIT) and zlib (zlib license), in `ext`, each with its license.

13.1 Use of AI tools

The code, the documentation, the benchmark and this manual were written with the help of an AI assistant, under the direction and review of the author. The logo and the illustration of the title page were made with AI image tools.